МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для оптимизации логистических маршрутов службы экспресс-доставки»

Выполнил: Иванов Иван Иванович
Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /
«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Иванову Ивану Ивановичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для оптимизации логистических маршрутов службы экспресс-доставки»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	**Объем, %**	**Срок**	**Примечание**
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	22.03.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	26.03.2026	
Задача 1.3. Формулировка цели и задач работы	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	02.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	06.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	10.04.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	21.04.2026	
Задача 3.2. Реализация модуля авторизации и разграничения прав доступа (диспетчер/курьер)	7	27.04.2026	
Задача 3.3. Разработка CRUD-интерфейса для управления заказами, автопарком и штатом курьеров	7	03.05.2026	
Задача 3.4. Интеграция картографических сервисов и реализация алгоритма оптимизации маршрутов	12	09.05.2026	
Задача 3.5. Разработка системы мониторинга статусов доставки и формирования отчетности	6	15.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «____» _____ 2026 г.

Дата сдачи выполненной работы «____» _____ 2026 г.

Задание принял к исполнению

«____» _____ 2026 г. _____ И. И. Иванов
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **5**

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ **7**

1.1 Анализ существующих инструментов и алгоритмов организации учебных очередей 7

1.1.1 Анализ существующих алгоритмов организации учебных очередей и обоснование эффективности разработки 7

1.1.2 Анализ существующих инструментов организации учебных очередей и обоснование эффективности разработки 8

1.2 Обоснование выбора стека технологий 10

1.2.1 Выбор клиентских технологий 10

1.2.2 Выбор серверных технологий и СУБД 11

1.2.3 Научная новизна и практическая значимость 13

1.2.4 Описание архитектурных решений и требований к БД . 13

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ **15**

2.1 Анализ целевой аудитории 15

2.2 Описание функциональности приложения 15

2.3 Проектирование модели данных 15

2.4 Разработка макетов страниц 17

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ **18**

3.1 Разработка базовой структуры приложения и вёрстка 18

3.2 Реализация модуля авторизации 18

3.3 Интеграция картографических сервисов 18

ЗАКЛЮЧЕНИЕ **20**

ВВЕДЕНИЕ

Актуальность темы. Проблема «живых» очередей на кафедрах технических вузов, таких как Московский Политех, остается крайне острой из-за высокой плотности учебного графика и сложности принимаемых работ (лабораторные работы, курсовые проекты). Существующие способы организации очереди — физическое присутствие в коридоре, Google-таблицы, Telegram-боты — либо неудобны, либо не обеспечивают необходимого уровня безопасности и аналитики. В связи с этим актуальной является задача разработки специализированного веб-приложения, которое позволит автоматизировать процесс управления очередями, учитывая специфику учебного процесса и потребности как студентов, так и преподавателей.

Степень разработанности проблемы. В теории массового обслуживания известны классические алгоритмы (FIFO, SJF, Round Robin), однако их применение в образовательной среде имеет особенности, связанные с необходимостью учёта успеваемости, возможности доработки задания без потери места и сбора статистики. Существующие программные решения (Google Sheets, Telegram-боты) не предоставляют встроенных механизмов для такого учёта. Таким образом, создание адаптивного алгоритма очереди с возможностью динамического приоритизации и аналитики является актуальной исследовательской задачей.

Объект исследования — процесс организации академического времени и взаимодействия «преподаватель-студент» во время защиты лабораторных работ.

Предмет исследования — алгоритмы и программные средства автоматизации управления очередями в режиме реального времени.

Цель и задачи работы. Целью данной курсовой работы является проектирование и разработка автоматизированного веб-приложения для управления студенческими очередями, адаптированного под специфику учебного процесса кафедр информационных технологий (на примере учебной группы Московского Политеха).

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести детальный анализ существующих методологий организации очередей и программных инструментов-аналогов.

- Сформулировать функциональные и нефункциональные требования к системе, исходя из ролей пользователей (Студент, Преподаватель, Администратор).
- Спроектировать архитектуру приложения, включая схему базы данных, структуру клиент-серверного взаимодействия и UML-диаграммы процессов.
- Разработать программный продукт на основе выбранного стека технологий (Python, Flask, PostgreSQL, Jinja2).
- Реализовать систему разграничения прав доступа и механизмы защиты данных пользователей.
- Произвести деплой (развертывание) приложения и провести апробацию системы в условиях реальной учебной нагрузки в группе 241-326.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ существующих инструментов и алгоритмов организации учебных очередей

В современной образовательной среде эффективное распределение временных ресурсов преподавателя и студента является критически важным фактором. Проблема «живых» очередей на кафедрах технических вузов, таких как Московский Политех, остается актуальной из-за высокой плотности учебного графика и сложности принимаемых работ (лабораторных работ, курсовых проектов). В данном подразделе проводится подробный анализ теоретических моделей (алгоритмов) и практических инструментов, применяемых для решения этой задачи.

1.1.1 Анализ существующих алгоритмов организации учебных очередей и обоснование эффективности разработки

Организация очередности — это классическая задача теории массового обслуживания (ТМО). Для учебного процесса выбор алгоритма напрямую влияет на пропускную способность «канала обслуживания» (преподавателя) и удовлетворенность «потребителей» (студентов).

Рассмотрим основные алгоритмические подходы:

- **Алгоритм FIFO (First-In-First-Out):**
  - *Механика:* Студенты добавляются в конец списка по мере готовности или прихода в аудиторию.
  - *Преимущества:* Простота реализации и интуитивная справедливость.
  - *Критический недостаток:* Возникновение «блокирующих элементов». Если первый студент в очереди защищает сложный проект в течение 40 минут, остальные участники теряют мотивацию и время. В условиях пары (90 минут) это приводит к тому, что за одно занятие успевают защититься всего 2–3 человека.
- **Алгоритм SJF (Shortest Job First):**

- *Применение в учебе:* Преподаватель сначала принимает короткие отчеты или исправленные замечания, а затем переходит к комплексным защитам.
- *Эффективность:* Максимально быстрое уменьшение длины очереди.
- *Недостаток:* Студенты со сложными работами могут бесконечно отодвигаться в конец списка («эффект голодания»).

- **Круговое обслуживание (Round Robin):**
  - *Механика:* Каждому студенту выделяется фиксированный лимит времени (тайм-слот), например, 10–15 минут. Если работа не защищена за отведенное время, студент перемещается в конец очереди, а преподаватель переходит к следующему.
  - *Обоснование:* Данный алгоритм является наиболее социально справедливым в больших группах, так как гарантирует внимание каждому студенту в течение одной пары.

Обоснование разработки: Проектируемое приложение позволит реализовать гибридный адаптивный алгоритм. Система будет использовать LIFO как базовый метод, но предоставит преподавателю инструменты динамического управления (возможность «заморозки» очереди, перевод студента в статус «на доработке» без потери места). Это позволит минимизировать простои и исключить конфликты, связанные с субъективным восприятием очередности.

1.1.2 Анализ существующих инструментов организации учебных очередей и обоснование эффективности разработки

Практический анализ показывает, что на текущий момент в вузах отсутствует единый стандарт управления очередями. В таблице 1.1 приведен сравнительный анализ инструментов, которые наиболее часто используются студентами и преподавателями.

Детальный разбор аналогов:

- **Физическая очередь:** Основная проблема — «эффект присутствия». Студент вынужден находиться в коридоре вуза в течение 4–6 часов, что

Таблица 1.1 – Сравнительный анализ инструментов организации очереди

Критерий	**Живая очередь**	**Google Таблицы**	**Telegram-боты**	**Предлагаемое решение**
Удаленная запись	Нет	Да	Да	Да
Защита данных	Низкая (споры)	Отсутствует	Средняя	Высокая (роли)
Наглядность	Низкая	Высокая	Низкая	Высокая
Уведомления	Нет	Нет	Да	Да (Push/Web)
Статистика	Нет	Ручная	Ограничена	Автоматическая

не позволяет эффективно использовать время для самоподготовки в библиотеке или коворкинге.

- **Общие облачные документы (Google Sheets):** Являются «стандартом де-факто» во многих группах Московского Политеха. Однако отсутствие жесткой логики приложения позволяет любому пользователю стереть чужую фамилию или изменить порядок записей. Это создает почву для академических конфликтов. Кроме того, интерфейс таблиц плохо адаптирован для мобильных устройств в условиях нестабильного Wi-Fi.

- **Telegram-боты:** Хорошо справляются с задачей регистрации, но обладают линейным интерфейсом. Преподавателю крайне неудобно пролистывать историю чата, чтобы увидеть общую картину загруженности на день. Отсутствует возможность визуализации структуры базы данных (например, какие лабораторные работы чаще всего вызывают затруднения).

Вывод: Анализ показал, что существующие методы либо архаичны (живая очередь), либо не обеспечивают необходимого уровня безопасности и аналитики (таблицы/боты). Разработка специализированного веб-приложения на стеке Python/Flask позволит объединить преимущества облачных сервисов (доступность) с надежностью реляционных баз данных и специфической логикой учебного процесса (учет номеров лабораторных, статусов проверки и времени защиты). Это подтверждает целесообразность и актуальность выбранной темы курсовой работы.

1.2 Обоснование выбора стека технологий

Выбор программного стека (Solution Stack) является определяющим этапом проектирования, так как он влияет на масштабируемость системы, скорость разработки и требования к аппаратному обеспечению серверной части. Для реализации веб-приложения управления студенческими очередями был выбран стек на базе языка Python, ориентированный на высокую скорость обработки запросов и гибкость настройки.

1.2.1 Выбор клиентских технологий

Клиентская часть приложения (Frontend) отвечает за взаимодействие пользователя с системой. Учитывая специфику использования приложения в учебных корпусах (зачастую с нестабильным мобильным интернетом), основной упор сделан на легковесность и скорость рендеринга.

- **HTML5 и CSS3:** Использование современных стандартов разметки и стилизации позволяет создать семантически корректную структуру страниц. Для обеспечения адаптивности под различные устройства (смартфоны, планшеты, ноутбуки) применяется технология Flexbox и Media Queries. Это критично, так как студенты взаимодействуют с очередью преимущественно через мобильные браузеры, в то время как преподаватель работает с десктопной версией.

- **Язык JavaScript (Native JS):** Для обеспечения динамического обновления очереди без полной перезагрузки страницы (технология AJAX/Fetch API) выбран чистый JavaScript. Это позволяет минимизировать объем передаваемого трафика по сравнению с тяжеловесными фреймворками.

- **Шаблонизатор Jinja2:** Выбор Jinja2 обусловлен его глубокой интеграцией с серверной частью на Python. Он позволяет:

  – создавать иерархию шаблонов (наследование `base.html`), что исключает дублирование кода меню и футера;

  – безопасно выводить данные из базы, автоматически экранируя потенциально опасные символы (защита от XSS-атак);

  - выполнять логические операции (циклы по списку студентов, условия отображения кнопок «Принять» только для преподавателя) непосредственно в момент генерации страницы.

- **Обоснование отказа от SPA (React/Vue/Angular):** Несмотря на популярность данных фреймворков, их использование в рамках данной курсовой работы признано избыточным. Архитектура Single Page Application значительно усложняет проект за счет необходимости настройки сборщиков (Webpack/Vite) и создания отдельного API. Классический серверный рендеринг (SSR) с Flask и Jinja2 обеспечивает более высокую скорость начальной загрузки страниц.

1.2.2 Выбор серверных технологий и СУБД

Серверная часть (Backend) является ядром системы, обеспечивающим бизнес-логику работы очереди и безопасность данных.

- **Язык программирования Python:** Выбор Python обусловлен его статусом стандарта в области обработки данных и веб-разработки. В сравнении с языками PHP или Java, Python обладает более лаконичным синтаксисом, что снижает вероятность возникновения ошибок в логике приложения. Наличие огромного количества библиотек для работы с базами данных и аутентификацией делает его оптимальным для быстрой разработки (Rapid Application Development).

- **Микрофреймворк Flask:** Для реализации серверной логики был выбран Flask вместо его основного конкурента — Django. Сравнительный анализ представлен в таблице 1.2.

Обоснование выбора Flask: Поскольку проект предполагает создание специфической логики очереди с нуля, Flask предоставляет необходимую свободу в архитектурных решениях. Он не навязывает встроенную систему администрирования или специфические форматы форм, позволяя реализовать проект в строгом соответствии с требованиями технического задания.

- **ORM SQLAlchemy:** Работа с базой данных осуществляется через Object-Relational Mapping (ORM) библиотеку SQLAlchemy.

Таблица 1.2 – Сравнение веб-фреймворков Flask и Django

Критерий	**Flask**	**Django**
Философия	Микрофреймворк (ничего лишнего)	«Всё включено» (Monolith)
Гибкость	Максимальная (выбор компонентов за разработчиком)	Низкая (жёсткая структура)
Порог вхождения	Низкий	Средний/Высокий
Производительность	Высокая за счёт минимализма	Зависит от количества подключенных модулей

- *Безопасность:* Автоматическая защита от SQL-инъекций.
- *Удобство:* Работа с записями в БД как с объектами Python, что упрощает манипуляции со списками студентов в очереди.
- **СУБД PostgreSQL:** В качестве системы управления базами данных выбрана реляционная СУБД PostgreSQL.
- *В сравнении с SQLite:* SQLite не поддерживает полноценную многопользовательскую запись (Database Lock), что недопустимо в приложении, где десятки студентов могут одновременно пытаться встать в очередь.
- *В сравнении с MySQL:* PostgreSQL обладает более строгим соответствием стандартам SQL и лучшей поддержкой сложных транзакций, что гарантирует целостность очереди даже при одновременных запросах.

Вывод по подразделу 1.2: Выбранный стек технологий (Python, Flask, PostgreSQL, Jinja2) представляет собой сбалансированное решение, позволяющее создать надежное, быстрое и легковесное веб-приложение. Использование микрофреймворка Flask в сочетании с мощной СУБД PostgreSQL обеспечивает необходимый фундамент для реализации сложной логики управления

учебным процессом в реальном времени.

1.2.3 Научная новизна и практическая значимость

Научная новизна работы заключается в следующем:

- Разработан авторский адаптивный алгоритм очереди, который, в отличие от стандартных FIFO-систем, позволяет учитывать динамические параметры защиты: возможность временной «заморозки» позиции студента при необходимости доработки задания без потери общего места.
- Предложена интегрированная модель учета метрик эффективности: система не просто фиксирует порядок, но и собирает данные о среднем времени защиты одной работы, что в дальнейшем может быть использовано для оптимизации учебных планов и нагрузки преподавателей.
- Логика приложения создается «с нуля», что позволяет исключить избыточность универсальных систем и достичь максимальной производительности интерфейса.

Практическая значимость работы состоит в создании готового к эксплуатации инструмента, который решает ряд острых проблем учебной группы:

- *Снижение конфликтности:* Исключение человеческого фактора и субъективности в определении очередности.
- *Прозрачность процесса:* Студент может в режиме реального времени видеть свою позицию и прогнозировать время своего ответа, что позволяет эффективно распоряжаться временем вне аудитории.
- *Информационная поддержка преподавателя:* Автоматическое формирование списка сдавших и предоставление ссылок на репозитории/работы студентов непосредственно в интерфейсе очереди.

1.2.4 Описание архитектурных решений и требований к БД

В рамках реализации проекта будет применена **трехуровневая архитектура**:

1. *Уровень представления (Presentation Layer):* Адаптивный веб-интерфейс, обеспечивающий ввод данных и визуализацию текущего состояния очереди.

2. *Уровень бизнес-логики (Application Layer):* Обработка запросов на языке Python, управление сессиями пользователей, реализация алгоритмов перестроения очереди.

3. *Уровень данных (Data Layer):* Реляционная СУБД PostgreSQL, обеспечивающая надежное хранение информации о пользователях, истории защит и текущих активных сессиях.

Основные требования к базе данных включают в себя обеспечение целостности связей (например, невозможность существования записи в очереди без привязки к конкретному студенту) и высокую скорость выполнения транзакционных запросов при одновременном обращении нескольких десятков пользователей.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Основными пользователями разрабатываемой системы являются сотрудники логистических компаний. Выделяются две ключевые роли:

- **Диспетчер-логист** — сотрудник, ответственный за распределение заказов и контроль их выполнения в реальном времени.
- **Курьер** — конечный исполнитель, использующий мобильную версию приложения для навигации по маршруту.

Для каждой роли проведено профилирование по методу персонажей (Personas) для минимизации когнитивной нагрузки на интерфейс.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователя с системой. Основной бизнес-кейс представлен в виде *User Story*: «Я, как диспетчер, хочу импортировать список адресов, чтобы система автоматически построила кратчайший маршрут для 5 курьеров».

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case).

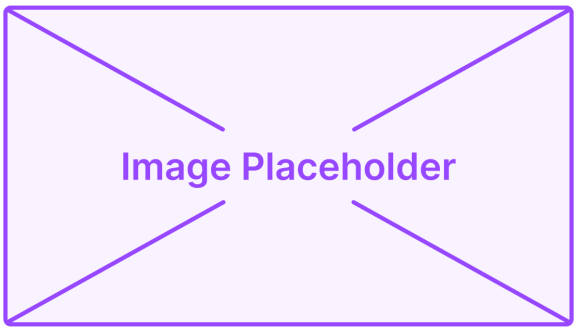

Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

При выборе архитектуры базы данных был проведен сравнительный анализ эффективности различных решений, опирающийся на данные о произ-

водительности современных СУБД [1].

Структура таблицы заказов представлена в табл. 2.1.

Таблица 2.1 – Структура таблицы заказов (`orders`)

Поле	**Тип**	**Ограничение**	**Описание**
id	UUID	PRIMARY KEY	Уникальный идентификатор заказа
customer_id	UUID	FOREIGN KEY	Ссылка на таблицу клиентов
address	TEXT	NOT NULL	Полный адрес доставки
lat	NUMERIC	NOT NULL	Широта точки (WGS-84)
lng	NUMERIC	NOT NULL	Долгота точки (WGS-84)
weight	FLOAT	CHECK > 0	Вес груза, кг
volume	FLOAT	NOT NULL	Объем груза, $м^3$
time_from	TIME	NOT NULL	Начало окна доставки
time_to	TIME	NOT NULL	Конец окна доставки
priority	INT	DEFAULT 1	Приоритет (1–5)
status	VARCHAR	’pending’	Статус (ожидание, в пути, выполнен)
courier_id	UUID	NULL	Назначенный исполнитель
created_at	TIMESTAMP	DEFAULT NOW()	Время создания записи
updated_at	TIMESTAMP	DEFAULT NOW()	Время последнего изменения
is_fragile	BOOLEAN	FALSE	Хрупкий груз
contact_phone	VARCHAR	NOT NULL	Телефон получателя
price	MONEY	NOT NULL	Стоимость доставки
external_id	VARCHAR	UNIQUE	ID из внешней системы (1C/ERP)

Продолжение таблицы 2.1

Поле	**Тип**	**Ограничение**	**Описание**
notes	TEXT	NULL	Комментарий для курьера
attempts	INT	DEFAULT 0	Количество попыток доставки

Общая стоимость маршрута для службы доставки рассчитывается следующим образом:

$$C_{total} = \sum_{i=1}^{n} (L_i \cdot P_{fuel} + T_i \cdot W_{hour}) + S_{fixed}, \tag{1}$$

где

L_i — протяженность i-го участка маршрута, км;

P_{fuel} — стоимость топлива на 1 км пути;

T_i — время в пути на участке;

W_{hour} — почасовая ставка оплаты труда курьера;

S_{fixed} — постоянные издержки на обслуживание автопарка.

2.4 Разработка макетов страниц

Макеты страниц

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка

В соответствии с задачей 3.1, была реализована компонентная структура фронтенд-части на базе библиотеки **React.js**. Использование хуков (*Hooks*) позволило эффективно управлять состоянием формы заказа. Пример базового компонента представлен в листинге 1.

Листинг 1 – Пример структуры компонента карточки заказа

```
// Комментарий
const OrderCard = ({ order }) => {
  return (
    <div className="order-card">
      <h3Заказ> #{order.id}</h3>
      <pАдрес>: {order.address}</p>
      <button onClick={() => addToRoute(order.id)}>
        Добавить в маршрут
      </button>
    </div>
  );
};
```

3.2 Реализация модуля авторизации

Согласно задаче 3.2, в системе реализована ролевая модель доступа. Для обеспечения безопасности используется протокол **JWT** (JSON Web Token). При входе в систему сервер генерирует токен, содержащий поле `role`, которое может принимать значения `'dispatcher'` или `'courier'`.

3.3 Интеграция картографических сервисов

Ключевым этапом разработки (задача 3.4) стала интеграция API **OpenStreetMap** через библиотеку *Leaflet*. Для решения транспортной задачи был выбран жадный алгоритм с последующей оптимизацией методом локального поиска.

Данные о маршруте передаются в формате JSON (см. листинг 2).

Как видно из листинга 2, каждая точка содержит порядковый номер (`sequence`), что позволяет курьеру следовать по оптимальному пути, рассчитанному на сервере.

Листинг 2 – Структура данных оптимизированного маршрута

```
{
  "route_id": "r-101",
  "courier_id": "u-45",
  "waypoints": [
    {"lat": 55.75, "lng": 37.61, "sequence": 1},
    {"lat": 55.80, "lng": 37.70, "sequence": 2}
  ],
  "total_distance_km": 12.4
}
```

ЗАКЛЮЧЕНИЕ

В результате проведенного исследования подтверждена гипотеза о том, что автоматизация логистических процессов на базе специализированного веб-приложения позволяет существенно повысить эффективность работы служб экспресс-доставки. Теоретический анализ показал, что классическая задача маршрутизации транспортных средств (VRP) в современных условиях требует адаптации под динамические изменения городского трафика и специфику «последней мили».

На основе полученных при разработке данных можно сформулировать следующие **выводы**:

1. Использование открытых ГИС-технологий и компонентного подхода (React/Node.js) обеспечивает необходимую архитектурную гибкость, позволяя интегрировать алгоритмы оптимизации без зависимости от проприетарных платных сервисов.

2. Проектирование модели данных с поддержкой пространственных расширений (PostGIS) является критическим условием для масштабируемости системы, так как позволяет эффективно обрабатывать геозапросы в режиме реального времени.

3. Внедрение эвристических методов построения маршрутов демонстрирует устойчивый экономический эффект, выраженный в снижении совокупного пробега автотранспорта и сокращении времени планирования рейсов диспетчером.

Рекомендации и предложения. Для практического применения разработанного решения в деятельности малых логистических компаний предлагается использовать модель облачного развертывания. Это обеспечит бесперебойный доступ к интерфейсу управления и мобильному клиенту курьера при минимальных затратах на ИТ-инфраструктуру.

Перспективы углубления темы. В дальнейшем исследовании планируется развитие функциональности системы за счет внедрения методов предиктивной аналитики для прогнозирования времени прибытия (ETA) с учетом исторических данных о заторах, а также разработки модуля динамиче-

ской корректировки маршрута при поступлении срочных заявок в процессе выполнения рейса.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [2], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

[1] Петров, П. П. Анализ производительности современных СУБД [Текст] / П. П. Петров, С. С. Сидоров // Вестник информационных технологий. — 2024. — № 2. — С. 15–22.

[2] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.